ShopKart E-Commerce Business Analytics

A complete SQL Data Analyst case study — from database design to strategic business intelligence — for an online marketplace operating across India.

CASE STUDY | SQL ANALYTICS | E-COMMERCE

Meet ShopKart

ShopKart is an online marketplace operating across major Indian cities, selling products across three core categories.

Electronics

Laptops, mobiles, headphones, and other high-value tech products driving significant revenue.

Fashion

Apparel including T-shirts and jeans — high volume, lower price-point category.

Home Appliances

Microwaves, refrigerators, air conditioners — large-ticket items with strong margins.

The Management Problem

Despite growing order volumes, ShopKart's leadership lacks clarity on the business's true financial health. Five critical gaps have been identified that require data-driven answers.

Profit Clarity

Sales are increasing, but profitability remains unclear. Management cannot distinguish high-revenue products from high-profit ones.

City-Level Anomalies

Certain cities show high order counts but surprisingly low revenue — a pattern that demands investigation.

Top Customer Identification

The business needs to identify its most valuable customers to enable targeted retention and loyalty strategies.

Category Performance

No clear view of which product categories are driving profit versus just volume.

Monthly Growth Tracking

A structured monthly order trend report is needed to understand growth trajectory over time.

👉 You are hired as the **SQL Data Analyst**. Your mission: analyse the database and deliver actionable insights to management.

Step 1 — Create the Database

The first step is to initialize the ShopKart database environment in MySQL. All subsequent tables and data will live within this database.

```
CREATE DATABASE shopkart_db;
USE shopkart_db;
```

This establishes the working schema. The `USE` statement ensures all subsequent DDL and DML commands are executed within the correct database context — a critical step before any table creation or data insertion.

Step 2 — Database Schema Overview

ShopKart's relational database is built on five interconnected tables. Each table has a clearly defined primary key, and foreign keys enforce referential integrity across the schema.

Customers table
Holds customer profiles and PK

Categories table
Product categories and hierarchy

Five-table Relational Schema

Orders table
Order header with FK to customers

Products table
Product catalog with FK to categories

Order Items table
Line items linking orders and products

The schema follows a classic star-like structure: **Orders** sits at the center, linked to **Customers** (who placed the order), **Order Items** (what was ordered), and **Products** (which belong to **Categories**). This design enables powerful multi-table JOIN queries for business analysis.

Step 3 — Insert Dataset

The following INSERT statements populate all five tables with ShopKart's sample dataset.

Customers

```
INSERT INTO customers VALUES
(1,'Rahul Sharma','Male','Delhi','2022-01-10'),
(2,'Priya Mehta','Female','Mumbai','2021-05-20'),
(3,'Amit Patel','Male','Ahmedabad','2023-02-14'),
(4,'Sneha Reddy','Female','Hyderabad','2022-11-01'),
(5,'Karan Verma','Male','Delhi','2020-08-18'),
(6,'Neha Nair','Female','Chennai','2021-09-09'),
(7,'Rohit Gupta','Male','Pune','2023-03-12'),
(8,'Anjali Singh','Female','Bangalore','2022-06-25');
```

Categories

```
INSERT INTO categories VALUES
(1,'Electronics'),
(2,'Fashion'),
(3,'Home Appliances');
```

Products

```
INSERT INTO products VALUES
(101,'Laptop',1,80000,65000),
(102,'Mobile',1,30000,22000),
(103,'Headphones',1,2000,1200),
(104,'T-Shirt',2,800,300),
(105,'Jeans',2,2000,900),
(106,'Microwave',3,7000,5000),
(107,'Refrigerator',3,30000,24000),
(108,'Air Conditioner',3,45000,35000);
```

Orders

```
INSERT INTO orders VALUES
(1001,1,'2024-01-10','Delhi','Delivered'),
(1002,2,'2024-01-11','Mumbai','Delivered'),
(1003,3,'2024-01-12','Ahmedabad','Cancelled'),
(1004,4,'2024-01-13','Hyderabad','Delivered'),
(1005,5,'2024-01-15','Delhi','Delivered'),
(1006,6,'2024-01-16','Chennai','Pending'),
(1007,1,'2024-02-02','Delhi','Delivered'),
(1008,7,'2024-02-05','Pune','Delivered'),
(1009,8,'2024-02-08','Bangalore','Delivered'),
(1010,2,'2024-02-10','Mumbai','Cancelled');
```

Order Items

```
INSERT INTO order_items VALUES
(1,1001,101,1),
(2,1001,103,2),
(3,1002,102,1),
(4,1003,104,3),
(5,1004,106,1),
(6,1005,105,2),
(7,1005,104,1),
(8,1006,102,1),
(9,1007,108,1),
(10,1008,103,3),
(11,1009,107,1),
(12,1010,104,2);
```

Table: Customers

Stores identity and demographic information for every registered ShopKart user. The `customer_id` is the primary key referenced by the Orders table.

```
CREATE TABLE customers (
  customer_id   INT PRIMARY KEY,
  customer_name VARCHAR(100),
  gender        VARCHAR(10),
  city          VARCHAR(50),
  signup_date   DATE
);
```

Sample Data — 8 customers across major Indian cities:

ID	Name	Gender	City	Signup Date
1	Rahul Sharma	Male	Delhi	2022-01-10
2	Priya Mehta	Female	Mumbai	2021-05-20
3	Amit Patel	Male	Ahmedabad	2023-02-14
4	Sneha Reddy	Female	Hyderabad	2022-11-01
5	Karan Verma	Male	Delhi	2020-08-18
6	Neha Nair	Female	Chennai	2021-09-09
7	Rohit Gupta	Male	Pune	2023-03-12
8	Anjali Singh	Female	Bangalore	2022-06-25

Tables: Categories & Products

Categories Table

```
CREATE TABLE categories (
  category_id   INT PRIMARY KEY,
  category_name VARCHAR(50)
);
```

Inserted Data:

- (1, 'Electronics')
- (2, 'Fashion')
- (3, 'Home Appliances')

Products Table

```
CREATE TABLE products (
  product_id   INT PRIMARY KEY,
  product_name VARCHAR(100),
  category_id  INT,
  price        DECIMAL(10,2),
  cost         DECIMAL(10,2),
  FOREIGN KEY (category_id)
    REFERENCES categories(category_id)
);
```

The `price` and `cost` columns are essential for all profit analytics. The foreign key to `categories` enables category-wise aggregation.

ID	Product	Category	Price (₹)	Cost (₹)
101	Laptop	Electronics	80,000	65,000
102	Mobile	Electronics	30,000	22,000
103	Headphones	Electronics	2,000	1,200
104	T-Shirt	Fashion	800	300
105	Jeans	Fashion	2,000	900
106	Microwave	Home Appliances	7,000	5,000
107	Refrigerator	Home Appliances	30,000	24,000
108	Air Conditioner	Home Appliances	45,000	35,000

Tables: Orders & Order Items

Orders Table

```
CREATE TABLE orders (
  order_id     INT PRIMARY KEY,
  customer_id  INT,
  order_date   DATE,
  city         VARCHAR(50),
  order_status VARCHAR(20),
  FOREIGN KEY (customer_id)
    REFERENCES customers(customer_id)
);
```

10 orders spanning January–February 2024. Statuses include **Delivered**, **Cancelled**, and **Pending** — enabling cancellation rate analysis.

Order Items Table

```
CREATE TABLE order_items (
  order_item_id INT PRIMARY KEY,
  order_id      INT,
  product_id    INT,
  quantity      INT,
  FOREIGN KEY (order_id)
    REFERENCES orders(order_id),
  FOREIGN KEY (product_id)
    REFERENCES products(product_id)
);
```

12 line items across 10 orders. This is the bridge table enabling revenue and quantity calculations by joining with `products`.

10
Total Orders
Across Jan–Feb 2024

7
Delivered
Successfully fulfilled orders

2
Cancelled
Orders 1003 & 1010

1
Pending
Order 1006 (Chennai)

LEVEL 1

Basic Exploration Queries

Level 1 establishes foundational SQL skills — filtering, sorting, and limiting results. These queries form the bedrock of any data investigation.

1

Customers from Delhi

Returns Rahul Sharma (ID 1) and Karan Verma (ID 5). Uses the `WHERE` clause to filter rows by city value.

2

Latest 5 Orders

Sorts all orders by `order_date` in descending order and returns only the 5 most recent. Useful for operational dashboards.

3

Top 3 Expensive Products

Returns Air Conditioner (₹45,000), Laptop (₹80,000), and Refrigerator (₹30,000) — the highest-priced items in the catalog.

LEVEL 2

Aggregation Thinking

Level 2 introduces aggregate functions and grouping — the core of business reporting. These queries answer "how many" and "how much" questions at scale.

1 Total Number of Orders

Uses `COUNT()` to return the total order volume. Result: **10 orders**. This is the simplest aggregation and a baseline KPI for any e-commerce business.

2 Total Quantity Sold Per Product

Groups order line items by product and sums quantities. Reveals which products move the most units — critical for inventory planning.

3 Cities with More Than 2 Orders

Uses `HAVING` (not `WHERE`) to filter after aggregation. Identifies high-activity cities — Delhi qualifies with 3 orders (1001, 1005, 1007).

LEVEL 3

Join-Based Real Analysis

Level 3 unlocks the true power of relational databases. By joining multiple tables, we can answer complex business questions that no single table can answer alone.

Q7 — Customer Name with Order ID

Joins `customers` + `orders` on the shared `customer_id` key. Produces a human-readable order list with customer names instead of raw IDs.

Q8 — Product Name in Each Order

A three-table JOIN spanning `orders`, `order_items`, and `products`. This is the most common pattern in e-commerce analytics — the order receipt view.

Q9 — Total Quantity Per Customer

Combines JOIN with `GROUP BY` to aggregate purchase volume per customer — a foundation for customer ranking and segmentation.

LEVEL 4

Profit Analytics

Profit analytics is one of the most industry-critical SQL skills. The key formula: **Profit = (Price − Cost) × Quantity**. These queries directly address management's core concern about unclear profitability.

Q10 — Profit Per Product

Calculates per-unit margin for every product. Air Conditioner leads at ₹10,000 profit per unit, followed by Laptop at ₹15,000 — revealing that Laptop is actually the highest unit-profit item.

Q11 — Category-Wise Total Profit

A four-table JOIN that rolls up profit to the category level. This answers the management question: which category is most profitable — Electronics, Fashion, or Home Appliances?

LEVEL 5

Customer Intelligence

Level 5 moves from transactional reporting to customer behavioral analysis — identifying repeat buyers and high-value purchasers using advanced SQL techniques.

Q12 — Customers with More Than 1 Order

Uses `HAVING COUNT` after grouping to isolate repeat customers. In the dataset, **Rahul Sharma** (orders 1001 & 1007) and **Priya Mehta** (orders 1002 & 1010) are the repeat buyers. Repeat customer rate is a key retention KPI.

Q13 — Customers Above Average Purchase Quantity

Introduces a **subquery** to dynamically compute the average purchase quantity and use it as a filter threshold. This is a hallmark of intermediate-to-advanced SQL — the inner query calculates the benchmark, the outer query applies it.

LEVEL 6

Strategic Business Questions

Level 6 delivers the executive-level insights that directly inform business strategy — city revenue rankings, product profitability, and growth trends over time.

1

Q14 — City with Highest Revenue

Addresses the management concern about cities with high orders but low revenue. Multiplies price × quantity per city and ranks them.

2

Q15 — Most Profitable Product

Combines margin calculation with quantity sold to find the product generating the most absolute profit — not just the highest price or volume.

3

Q16 — Monthly Order Trend

Uses `MONTH()` date function to extract the month from each order date and group by it. Produces the monthly growth report management requested — January vs. February comparison.

Final Capstone Task — Complete Analyst Simulation

The manager's final requirement: build a comprehensive SQL report that consolidates all analytical dimensions into a single business intelligence deliverable. This is the **Complete Data Analyst Simulation Project**.

Top 3 Customers by Purchase

Rank customers by total quantity or revenue purchased.

Revenue Per City

Aggregate price × quantity grouped by city to expose geographic revenue distribution and flag underperforming markets.

Profit Per Category

Four-table JOIN rolling up (price − cost) × quantity to category level — the definitive category P&L view.

Repeat Customer Count

Count distinct customers with more than one order after grouping by customer.

Cancelled Order %

Calculate cancelled divided by total orders × 100 using conditional aggregation or a subquery to derive the cancellation rate.

Project Summary — Skills Progression

This case study covers the full spectrum of SQL skills required for a professional data analyst role, progressing from basic syntax to strategic business intelligence.

1	Level 1	WHERE, ORDER BY, LIMIT
2	Level 2	COUNT, SUM, GROUP BY, HAVING
3	Level 3	Multi-table JOINs
4	Level 4	Profit Analytics (price − cost)
5	Level 5	Subqueries & Customer Intelligence
6	Capstone	Full Business Simulation Report

By completing all 16 questions and the capstone task, you will have demonstrated the ability to design a normalized relational schema, populate it with realistic data, and extract multi-dimensional business insights — the core competency of a **SQL Data Analyst**.